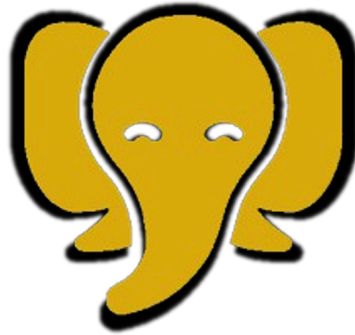


MED

Memory Evasion Detection



by

Elad Zinkin

Itay Lugasi

Idan Manbar

Eyal Schuldenfrei

Approved by the supervisor: Dr. Hemi Leibowitz

Submitted to the Computer Science Faculty of College of Management

September 2024

Rishon LeZion

Acknowledgments

We would like to thank the Computer Science Faculty of the College of Management for their support during our three years of study. Their help was crucial in the process of researching and developing the MED project. We also appreciate their efforts in organizing the projects day.

Special thanks to Dr. Hemi Leibowitz for guiding us throughout the project and providing valuable advice.

Executive Summary

This project, MED (Memory Evasion Detection), focuses on the detection of sleep obfuscators, a sophisticated malware evasion technique that delays execution to avoid detection by traditional security tools. The primary objective of MED is to address the limitations of existing tools, such as Windows Defender, VirusTotal, and the Volatility Framework's 'malfind' plugin, which struggle to detect sleep obfuscation techniques.

MED is implemented in Python, leveraging the Volatility[1] framework and a custom plugin that integrates the Unicorn[2] library to emulate CPU instructions. This allows MED to thoroughly scan memory for malware that utilizes sleep obfuscators. The system is built to be scalable, with a modular design that allows easy extension of its functionalities.

The experimental methodology involved running malware samples within isolated virtual machines[3] (VMs), ensuring a secure environment for thoroughly investigating their techniques. This setup allowed us to pinpoint critical detection points and test MED's effectiveness in both static and live environments. MED's key capabilities include extracting malicious data and shellcode, consistently identifying sleep obfuscators that traditional tools fail to detect.

The results demonstrate that MED excels in detecting sleep obfuscators across all test cases, outperforming established tools like VirusTotal and Windows Defender, which often fail to recognize these subtle evasion techniques. MED provides a specialized solution for malware analysis in environments where sophisticated evasion tactics are increasingly common, filling a critical gap in existing detection methodologies.

In conclusion, MED successfully achieves its objective of identifying sleep obfuscation techniques, providing a valuable tool for more comprehensive malware detection.

Table of Contents

1. Introduction	7
1.1. Background	7
1.2. Problem Statement	7
1.3. Objectives	7
1.4. Scope and Limitations	8
1.5. Methodology	8
1.6. Organization of the Project Book	8
2. Literature Review	10
2.1. Memory Management in Windows: An In-Depth Overview	10
2.2. Malware	14
2.3. Sleep Obfuscation	16
2.4. Memory Scanners	18
2.5. Malware Investigation	21
2.6. Virtualization in Malware Analysis	24
2.7. Memory Acquisition Tools	25
2.8. Overview of Relevant Literature	27
3. Research	28
3.1. Malware Research	28
3.2. Common factor	36
3.3. Offline Solution	36
3.4. Live Solution	37
4. System Design and Implementation	38
4.1. System Architecture	38
4.2. Implementation Details	39
4.3. Evaluation Metrics	41
5. Results and Analysis	43
5.1. Experimental Setup	43
5.2. Presentation of Results	45
5.3. Data Analysis and Interpretation	49
5.4. Comparison with Existing Approaches	50
5.5. Discussion of Findings	52
6. Conclusion and Future Work	53
7. References	54
8. Appendix A	55
A.1 Project Setup Instructions	55

Table of Abbreviations

MED - Memory Evasion Detection

AV - AntiVirus

VM - Virtual Machine

VT - VirusTotal

DFIR - Digital Forensics and Incident Response

Table of Figures

Figure 1: Gargoyle is executing and running its payload.....	28
Figure 2: Gargoyle page protection Read.....	29
Figure 3: Gargoyle page protection Read Execute.....	29
Figure 4: Cronos is executing and running its payload.....	32
Figure 5: Cronos memory while executing.....	32
Figure 6: Cronos memory while sleeping.....	33
Figure 7: Building thread contexts, one per stage.....	34
Figure 8: Timer initialization of each stage.....	35
Figure 9: General sleep-obfuscator execution flow.....	36
Figure 10: General sleep-obfuscator execution flow vs MED's algorithm.....	37
Figure 11: Complete Architecture.....	39
Figure 12: 'Screen Main' inherits ScreenMgmt abstract class.....	40
Figure 13: VT integration.....	40
Figure 14: Memory Peeking.....	41
Figure 15: VirusTotal results on Cronos binary and shellcode.....	42
Figure 16: VirusTotal results on Gargoyle binary and shellcode.....	42
Figure 17: Working with VM snapshots.....	43
Figure 18: Live scan detecting Gargoyle.....	46
Figure 19: Static scan detecting Cronos.....	46
Figure 20: Running a scan on a clean machine.....	48
Figure 21: Detection Comparison.....	50

1. Introduction

1.1. Background

In today's world of advanced malware and memory manipulation techniques, security professionals are continually challenged by attackers who employ sophisticated methods to evade detection. Memory evasion techniques have become a critical area of focus for security researchers, as they enable malicious actors to remain hidden within a system for extended periods. Unlike traditional file-based malware, which can often be detected through signature-based scanning or heuristic analysis, memory-resident malware operates in a more elusive manner. By manipulating system memory, attackers can bypass conventional detection mechanisms, making it difficult to identify their presence through standard security tools. These techniques often exploit weaknesses in real-time monitoring and forensics, presenting a significant challenge in the field of DFIR.

1.2. Problem Statement

Memory evasion techniques pose a significant challenge for DFIR teams. Existing tools often struggle to detect or address the increasingly sophisticated strategies employed by attackers to hide malicious activity in system memory. Current research and detection methods have yet to provide a comprehensive solution for identifying memory evasion behaviors in a broad and generic manner. This project seeks to address this gap by developing a tool designed to detect various forms of memory evasion within system memory.

1.3. Objectives

The main objective of the MED project is to design and implement a generic solution that can detect memory evasion techniques, with a specific focus on sleep obfuscation. Key objectives include:

1. Investigating common memory evasion strategies.
2. Developing a new Volatility plugin to detect various forms of sleep obfuscation.
3. Creating a command-line tool that integrates the plugin and presents findings in a user-friendly manner.

4. Testing and validating the tool against real-world malware samples that employ sleep obfuscation.

1.4. Scope and Limitations

The scope of this project is focused on the detection of sleep obfuscation techniques in memory, specifically for Windows-based systems. While the methods explored may be relevant to other memory evasion techniques, the primary emphasis remains on identifying altered sleep patterns used by malware. The tool supports both live memory scanning and post-mortem analysis, enhancing its flexibility in detecting threats in real time as well as during forensic investigations. This project leverages the Volatility framework for memory forensics, which brings its own set of limitations, such as compatibility with specific file formats and operating systems.

1.5. Methodology

The methodology of this project involves the following steps:

1. **Literature Review:** Research existing memory evasion techniques, with a focus on sleep obfuscation.
2. **Tool Selection:** Volatility is chosen as the primary tool for memory analysis due to its extensibility and widespread use in the field.
3. **Development:** A new Volatility plugin is developed to detect sleep obfuscation, incorporating detection strategies such as analyzing suspicious timers and sleep-related API calls.
4. **Testing:** The tool is tested against several malware samples known to employ sleep obfuscation, and results are analyzed to determine the effectiveness of the plugin.

1.6. Organization of the Project Book

This project book is organized into the following chapters:

- **Chapter 1: Introduction** – Provides an overview of the project, including background, problem statement, objectives, scope, methodology, and organization of the book.
- **Chapter 2: Literature Review** – Summarizes existing research and tools related to memory evasion techniques, with a focus on detection strategies.
- **Chapter 3: Research** – Details the process of researching the different mechanisms and developing the solution.
- **Chapter 4: System Design and Implementation** – Details the design and development of the MED tool, including the Volatility plugin and its functionality.
- **Chapter 5: Results and Analysis** – Presents the results of testing the tool against malware samples and discusses the analysis of those results.
- **Chapter 6: Conclusion and Future Work** – Concludes the project, summarizing key findings and suggesting potential areas for future development and research.
- **References** – Lists all sources cited throughout the project.

2. Literature Review

2.1. Memory Management in Windows: An In-Depth Overview

Memory management is a fundamental aspect of operating systems, and Windows is no exception. Efficient memory management ensures that applications run smoothly, resources are allocated effectively, and overall system performance is optimized. In this article, we will explore the intricacies of memory management in Windows, covering key concepts, mechanisms, and best practices

2.1.1. Definition and Importance

Memory management refers to the process of coordinating and handling computer memory resources. It involves the allocation, usage, and release of memory space for applications and system processes.

Proper memory management is vital for:

System Stability: Preventing memory leaks and fragmentation, which can lead to crashes and slowdowns.

Performance Optimization: Ensuring applications run efficiently without unnecessary delays.

Resource Sharing: Allowing multiple applications to coexist and share memory without conflict.

Security: Protecting processes from unauthorized access and malicious activities. Memory management includes mechanisms such as memory protection and address space isolation, which prevent one process from reading or modifying another process's memory space. This isolation is crucial for safeguarding sensitive data and maintaining the integrity of the operating system.

2.1.2. Virtual Memory vs. Physical Memory

Windows employs a virtual memory management system that abstracts physical memory (RAM) to provide a more flexible and efficient memory allocation process. Virtual memory allows the system to use disk space as an extension of RAM, enabling applications to use more

memory than is physically available.

Advantages of using virtual memory

- It can handle twice as many addresses as main memory.
- It enables more applications to be used at once.
- It frees applications from managing shared memory and saves users from having to add memory modules when RAM space runs out.
- It has increased speed when only a segment of a program is needed for execution.
- It has increased security because of memory isolation.
- It enables multiple larger applications to run simultaneously.
- Allocating memory is relatively inexpensive.
- It does not need external fragmentation.
- And more.

Some of the above advantages will be covered and illustrated later, as we will focus mainly on the security aspects of virtual memory.

2.1.3. Address Space

Each process in Windows operates within its own virtual address space, typically ranging from 0x00000000 to 0xFFFFFFFF (4 GB for 32-bit systems, 8 TB for 64-bit systems). This separation prevents processes from interfering with one another, enhancing security and stability.

2.1.4. Memory Allocation

Windows uses various memory allocation techniques to manage how memory is distributed to applications:

Heap Allocation: The Windows heap manager allocates memory for dynamic memory requests from applications. It supports multiple heaps, allowing for different memory allocation strategies.

Stack Allocation: Each thread in Windows has its own stack, used for function calls and local variables. The stack grows and shrinks as functions are called and returned.

2.1.5. Paging

Windows utilizes a paging mechanism to manage virtual memory. The virtual address space is divided into fixed-size pages, typically 4 KB. When a process requires more memory than what is physically available, the system moves less frequently accessed pages to a page file on disk, freeing up RAM for active processes.

2.1.6. Page Tables

To translate virtual addresses to physical addresses, Windows employs page tables. Each process has its own page table that maintains the mapping of virtual pages to physical frames in RAM. Besides the virtual-to-physical address translation, page tables store additional information crucial for memory management, including:

- **Access Permissions:** Each entry contains flags indicating the permissions associated with the page, such as read, write, and execute rights. This ensures that processes can only access memory areas in the way they are authorized to.
- **Present/Absent Flags:** These flags indicate whether the corresponding physical page is currently loaded in RAM or needs to be fetched from disk. This is essential for handling page faults effectively.
- **Dirty Bit:** This bit tracks whether a page has been modified since it was last loaded into memory. If a page is dirty when it's evicted, it must be written back to disk, ensuring data consistency.
- **Referenced Bit:** This bit is set when a page is accessed. It helps the operating system determine which pages are actively used and which can be swapped out to optimize memory usage.

This comprehensive information allows Windows to efficiently manage memory, optimize performance, and maintain process isolation and security.

2.1.7. Virtual Address Descriptors (VADs)

Windows also uses Virtual Address Descriptors (VADs) to manage memory regions allocated to processes. Each VAD represents a contiguous block of virtual memory and contains information such as the starting and ending addresses, protection attributes, and the status of the memory (e.g., committed, reserved, or free). VADs are crucial for maintaining the integrity of memory management and ensuring proper access controls.

2.1.8. Memory Protections

Memory protection mechanisms in Windows prevent unauthorized access to memory regions. This is crucial for maintaining process isolation and security. Each memory page can have specific access rights, such as read, write, or execute permissions. When a process attempts to access memory outside its allowed permissions, a protection fault occurs, and the operating system can terminate the offending process to maintain system stability.

2.1.9. VAD Protections vs Page Table Protections

Maybe you noticed that both VADs and page tables store memory protection data.

However, it is important to emphasize some key differences between them:

- **Different purpose** - Reminder: every page table entry describes one page in memory, while every VAD describes a contiguous block of virtual memory (consisting of various pages with similar attributes). In other words, VAD protections are only part of a high level data on a memory region, and they are not used by the OS to enforce security policies, while page tables protections are indeed used by the operating system to enforce security policies at the page level, ensuring that only authorized processes can access or modify particular pages of memory.
- **Up to date protections** - While page table permission are being updated every time there is a change (e.g. when VirtualProtect is called to change a page's permission), the VAD permission **won't** get updated and it would store only the initial protections that were used when the memory region was allocated.

The reason for this behavior is the difference in the purpose of VAD and page table protections, as described in the previous point.

2.1.10. Page Faults

A page fault occurs when a program tries to access a page that is not currently in physical memory. Windows handles page faults efficiently by loading the required page from disk into RAM. There are two types of page faults:

- **Minor Page Faults:** The required page is found in the page cache.
- **Major Page Faults:** The page must be loaded from the disk, which is slower and can lead to performance degradation.

2.2. Malware

Malware, or malicious software, refers to any software program or code developed with the intent to harm, exploit, or compromise a system, network, or device. It can take various forms, including viruses, worms, trojans, spyware, ransomware, and rootkits, each designed for different malicious purposes. The defining feature of malware is its intent to perform unauthorized actions, such as stealing data, corrupting files, or gaining control over systems. It infiltrates systems via various means, including phishing emails, malicious websites, or exploiting software vulnerabilities, often operating covertly to evade detection by traditional security tools.

2.2.1. Malware Behaviors and Goals

Malware can be programmed to achieve a wide range of malicious objectives. Some malware is designed to destroy data or render systems unusable, while others aim to steal sensitive information, such as passwords or financial data. The primary goals of malware include:

- **Data Exfiltration:** Malware like spyware and keyloggers capture confidential data and send it to attackers.
- **System Disruption:** Some malware, like ransomware and viruses, can damage system functionality by encrypting files or corrupting essential processes.
- **Unauthorized Access:** Malware often facilitates backdoor access to systems, allowing attackers to control compromised machines remotely.

- **Financial Gain:** A significant amount of malware, particularly ransomware and banking trojans, is designed to generate direct financial returns for cybercriminals by demanding ransoms or stealing banking credentials.

Many advanced malware variants also attempt to remain persistent within systems, running undetected for extended periods to maximize the damage or data theft they can achieve.

2.2.2. Memory-based Malware

Memory-based malware, also known as fileless malware, is designed to operate directly within a system's memory without leaving traditional file traces on the hard drive. Unlike conventional malware that relies on executable files, memory-based malware injects its code directly into the memory space of legitimate running processes, making it significantly harder to detect. It leverages the system's existing resources, avoiding detection by file-based antivirus scanners.

Memory-based malware often exploits vulnerabilities in legitimate processes, such as browsers, document readers, or system services, to gain execution in memory. It typically disappears once the system is rebooted, making it highly stealthy. However, because it operates entirely in memory, it leaves traces in memory dumps, which can be analyzed using memory forensics tools like Volatility.

2.2.3. Evasion Techniques

Malware has evolved over time to become more sophisticated, using a variety of evasion techniques to avoid detection by security software and investigators. Some of the most common evasion methods include:

Code Injection: Malware can inject its code into legitimate processes, making it harder to detect. Examples include process hollowing, where malware replaces the contents of a legitimate process in memory, and DLL injection, where malicious code is injected into the dynamic link libraries of other processes.

Packing and Encryption: Malware authors often use packing techniques to compress or encrypt their malware, hiding the true nature of the code until it is unpacked and executed. This technique helps malware evade signature-based detection methods.

API Hooking: By intercepting and manipulating API calls, malware can alter the behavior of legitimate processes or hide itself from security tools. Hooking techniques allow malware to modify the way the operating system interacts with applications.

Timing and Execution Delays (Sleep Obfuscation): Sleep obfuscators introduce delays in execution to evade automated analysis tools, such as sandboxes. By waiting for extended periods before executing malicious code, malware can outlast the analysis environment's runtime and avoid detection. This is particularly effective in environments that monitor activity for only a short period.

2.3. Sleep Obfuscation

Sleep obfuscation is a technique used by malware to delay its execution, making it harder for security tools and researchers to detect its presence. By introducing intentional pauses or “sleep” cycles into its code, malware can avoid being flagged by automated analysis systems that monitor suspicious activity over a limited timeframe. During these sleep periods, the malware remains dormant and does not perform any malicious actions, thus escaping detection. Once the delay ends, the malware resumes its operation, often launching its attack when the monitoring tools have ceased their activity.

2.3.1. Common Uses of Sleep Obfuscation

Sleep obfuscation is commonly used by advanced malware to evade security measures. Some of the most frequent applications include:

Sandbox Evasion: Many automated malware analysis environments, like sandboxes, run for a limited amount of time (e.g., a few minutes). Sleep obfuscators delay the execution of malicious payloads until after the sandbox stops monitoring, allowing the malware to bypass detection.

Bypassing Real-Time Monitoring: Security tools that monitor for immediate malicious behavior may miss malware that uses long sleep cycles. This allows the malware to remain under the radar of real-time protection mechanisms.

Persistent Threats: In more complex attacks, malware may use sleep obfuscation to hide for extended periods, activating only after a specific trigger, such as a network connection or user activity. This allows malware to persist within a system without alerting security measures.

2.3.2. Techniques for Detecting Sleep Obfuscators

Detecting malware that uses sleep obfuscation requires specialized approaches, as traditional methods often fall short. Some of the techniques used to identify sleep obfuscators include:

Memory Forensics: Memory analysis tools like Volatility can capture and analyze live memory states, allowing investigators to detect dormant malware that has not yet activated. Since sleep obfuscators leave traces in system memory even when inactive, memory forensics can be an effective method for uncovering these threats.

Extended Dynamic Analysis: Instead of analyzing malware within a short time window, sandboxes and other analysis tools can extend the runtime of the environment, increasing the likelihood of observing malicious actions after sleep delays.

Heuristic-Based Detection: Some security tools use heuristics to detect patterns in code that resemble sleep obfuscators. These tools look for functions or calls to APIs such as `Sleep()`, `NtDelayExecution()`, or `WaitForSingleObject()`, which are often used by malware to implement sleep obfuscation.

2.3.3. Limitations of Traditional Detection Tools

Traditional security tools, such as antivirus software and basic memory scanners, are generally not equipped to detect sleep obfuscators due to their reliance on real-time monitoring and signature-based detection. The key limitations of these tools include:

Limited Timeframe: Many detection tools and sandboxes operate under tight time constraints, monitoring for only a few minutes before terminating the analysis. This allows sleep obfuscators, which delay malicious activity, to avoid detection.

Signature-Based Detection: Traditional antivirus solutions rely heavily on signature-based methods, which look for known patterns of malicious code. Sleep obfuscators, by delaying their execution, often evade detection because they do not exhibit any malicious behavior during the

scanning period. Moreover, some of them perform obfuscation or encryption of the code while they don't execute, making the signature-based detection ineffective.

Inability to Handle Delays: Most traditional tools do not account for extended periods of inactivity. They are designed to flag immediate threats, making them ineffective against malware that purposely delays its actions.

Lack of Memory Analysis: Many traditional tools do not perform deep memory analysis, which is required to detect malware that resides solely in memory or uses obfuscation techniques. Without this capability, sleep obfuscators can remain hidden within the system's memory.

2.4. Memory Scanners

Memory scanners are crucial in detecting advanced threats like fileless malware, which reside in system memory rather than on disk. These tools analyze active memory to uncover hidden processes and unauthorized code injections that evade traditional antivirus solutions. This chapter explores key memory scanning tools, including the Volatility Framework, and highlights their strengths and limitations in addressing modern malware challenges.

2.4.1. Volatility Framework

The **Volatility Framework** is one of the most widely used tools for memory forensics and analysis. It is an open-source platform that allows investigators to analyze memory dumps for malicious activity. Volatility supports multiple operating systems, including Windows, Linux, and macOS, and provides a wide range of plugins for detailed memory analysis. Some of its key functionalities include process enumeration, DLL and handle tracking, and network connection analysis.

- **Malfind Plugin:** One of Volatility's most notable plugins, **malfind**, is used to detect hidden or injected processes, which are common signs of malware activity. It helps analysts locate suspicious memory regions where malware may be injected.
- **Memdump Plugin:** This plugin extracts memory segments to a file, enabling deeper offline analysis of potential threats.

- **Handles Plugin:** This plugin reveals open file handles and objects in memory, which can be key indicators of unauthorized access or manipulation by malware.

Volatility is highly extensible, allowing custom plugins to be developed for more specialized tasks. In particular, its flexibility makes it a key tool for detecting malware that operates directly in memory, such as sleep obfuscators and fileless malware.

2.4.2. Volatility's Malfind Plugin

Malfind is a volatility plugin that is used to find hidden and injected code.

How?

It parses all the VAD trees in the operating system, locating all those that have EXECUTE and WRITE protections. Then it performs some whitelisting (e.g. if the memory is a shared memory and the protections are PAGE_EXECUTE_WRITECOPY, it means that it is a shared library with no changes, hence we can ignore it).

2.4.3. Volatility's Malfind Plugin Weaknesses

Malfind has some weaknesses that are also present in other memory scanners, which we will discuss later.

However, one significant limitation specific to Malfind is its reliance on the Virtual Address Descriptor (VAD) tree for obtaining memory protection information. It's important to note that VAD protections represent only the initial protections.

Therefore, if malware allocates a memory region with protections such as NO_ACCESS and later modifies those protections to include EXECUTE and WRITE, Malfind will not detect this change, as the VAD protections would still reflect the original NO_ACCESS status.

2.4.4. Other Memory Scanning Tools

In addition to Volatility, there are several other memory scanning tools commonly used in memory forensics:

- **Rekall:** Rekall is another open-source memory forensics tool that offers similar functionality to Volatility. It is designed for memory analysis across multiple operating

systems, and it emphasizes speed and modularity. Rekall provides plugins for analyzing memory dumps, processes, and kernel structures.

- **Redline:** Developed by FireEye, Redline is a commercial tool for memory analysis and malware detection. It is user-friendly and provides a graphical interface, making it accessible to investigators who are less familiar with command-line tools. Redline focuses on identifying rootkits, APTs, and other malware in memory, providing forensic investigators with a streamlined approach to incident response.
- **FTK Imager:** Though primarily a disk imaging tool, FTK Imager can also capture live memory. It is often used in combination with other tools like Volatility or Redline for memory analysis. FTK Imager allows investigators to create memory snapshots, which can then be examined in greater detail with specialized tools.
- **Memdump (Linux):** On Linux systems, memdump is used to capture raw memory. While it lacks advanced analysis capabilities, it is often used as part of a larger workflow, enabling investigators to create memory snapshots that can be examined using more advanced tools like Volatility or Rekall.

2.4.5. Strengths and Weaknesses of Current Memory Scanners

Memory scanners play a crucial role in detecting sophisticated malware that may not leave a signature on disk. However, while these tools are essential for memory forensics, they have their strengths and weaknesses:

Strengths:

- **Detecting Fileless Malware:** Tools like Volatility and Rekall are effective at uncovering fileless malware, which operates exclusively in memory and leaves no trace on the disk. By analyzing active processes and memory regions, these tools can identify malware that would otherwise go unnoticed.
- **In-depth Memory Analysis:** Memory scanners allow forensic investigators to perform a detailed analysis of system memory. They can detect hidden processes, suspicious network connections, and unauthorized modifications to the kernel.
- **Customizable and Extensible:** Open-source tools like Volatility and Rekall can be customized to meet specific investigative needs. This flexibility allows researchers to develop plugins for new malware variants or forensic techniques.

- **Platform Independence:** Many memory scanners, including Volatility, support multiple platforms, making them versatile for analyzing systems running Windows, Linux, and macOS.

Weaknesses:

- **Steep Learning Curve:** Tools like Volatility, while powerful, require a deep understanding of memory structures and operating system internals. This makes them difficult for novice investigators to use effectively.
- **Limited Real-Time Detection:** Most memory scanners are post-mortem analysis tools, meaning they require a memory dump from a live system or captured snapshot. They do not typically provide real-time malware detection, which limits their effectiveness in environments requiring rapid response.
- **Performance Overhead:** Memory scanning, especially for large dumps, can be resource-intensive and time-consuming. Scanning large amounts of memory data may take significant time, which can be an issue in time-sensitive investigations.
- **Complexity in Large Environments:** In large-scale environments with many machines, manually analyzing memory dumps from each machine can be a daunting task. Automation is possible but often requires additional scripting or integration with other security tools, increasing the complexity of the process.

2.5. Malware Investigation

Malware investigation is essential in cybersecurity, focusing on identifying, containing, and eradicating threats. It involves techniques like behavioral analysis and memory forensics to uncover malicious activity. Collaboration with threat intelligence platforms enhances detection and response efforts. This chapter explores the key processes and tools used in effective malware investigation.

2.5.1. Incident Response Process

The **Incident Response Process** is a structured approach to handling security breaches and cyberattacks. It involves a series of steps aimed at quickly detecting, responding to, and recovering from an incident while minimizing the damage to the organization. The typical incident response process includes:

- **Preparation:** Establishing policies, procedures, and tools in advance to effectively respond to incidents.
- **Identification:** Detecting the security breach, often through monitoring systems, intrusion detection tools, or alerts from users.
- **Containment:** Taking immediate action to limit the impact of the attack, such as isolating affected systems to prevent further spread.
- **Eradication:** Identifying the root cause of the incident and removing malicious components from the affected systems.
- **Recovery:** Restoring systems to normal operation and ensuring that vulnerabilities exploited by the attack are addressed.
- **Lessons Learned:** After the incident is resolved, analyzing the response and identifying areas for improvement in the security strategy.

Memory forensics plays a critical role during the identification, containment, and eradication phases by providing deeper insight into malicious activity that traditional disk forensics might miss.

2.5.2. Behavioral Analysis of Malware

Behavioral analysis focuses on observing and understanding how malware behaves once executed, rather than relying on static signatures or code analysis. This dynamic approach involves running malware in a controlled environment, such as a virtual machine or sandbox, and monitoring its interactions with the system. Key behaviors that are analyzed include:

- **System Modifications:** Tracking changes to system files, processes, and registry keys.
- **Network Activity:** Observing outgoing connections or communications with command-and-control servers.
- **Persistence Mechanisms:** Identifying how malware ensures it remains on the system, such as through scheduled tasks or startup entries.
- **Memory Activity:** Analyzing how malware interacts with system memory, whether by injecting code into legitimate processes or manipulating memory structures.

Behavioral analysis is particularly effective in understanding complex, polymorphic, and obfuscated malware, as it captures the malware's actions in real time.

2.5.3. Role of Memory Forensics in Malware Investigation

Memory forensics is a crucial part of malware investigations, especially when dealing with advanced threats that operate primarily in memory. Since some malware does not leave traces on the file system, memory forensics provides visibility into malicious activity by analyzing the system's volatile memory. Key roles of memory forensics include:

- **Detecting In-Memory Malware:** Malware that does not create files on disk can still be captured in memory dumps, allowing investigators to uncover malicious processes or injected code.
- **Uncovering Hidden Processes:** Memory forensics tools like Volatility can reveal hidden or terminated processes that standard monitoring tools may miss.
- **Identifying Network Connections and Activity:** Investigators can analyze memory to detect ongoing or past network communications with malicious servers.
- **Recovering Encrypted or Obfuscated Data:** Malware often operates by encrypting its code to avoid detection. Memory forensics can capture the malware while it is executing and decrypted in memory.

By providing this level of detailed insight, memory forensics enables investigators to build a comprehensive picture of the attack, even when traditional forensic techniques fall short.

2.5.4. Collaboration with Threat Intelligence Platforms

Collaborating with **Threat Intelligence Platforms** enhances malware investigation by leveraging shared data on known threats and indicators of compromise (IOCs). Threat intelligence provides context, allowing investigators to correlate suspicious behaviors and artifacts with known malware families and attack patterns. Benefits of integrating threat intelligence into malware investigation include:

- **Identifying Known Threats:** By cross-referencing IOCs in the memory dump with threat intelligence databases, investigators can quickly identify whether the malware is part of a larger, known campaign.
- **Enhancing Detection:** Threat intelligence feeds can be integrated into forensic tools to enhance detection capabilities, especially for new or evolving threats.

- **Providing Context for Attacks:** Intelligence platforms offer context around specific malware's behavior, goals, and common tactics, techniques, and procedures (TTPs), helping investigators better understand the threat they are dealing with.
- **Real-Time Threat Monitoring:** Many organizations subscribe to threat intelligence platforms to receive real-time updates about emerging threats. This allows security teams to respond faster and update their incident response strategies based on the latest intelligence.

Incorporating threat intelligence into the malware investigation process strengthens the ability to detect and respond to advanced threats in a timely manner.

2.6. Virtualization in Malware Analysis

Virtual machines (VMs) play a crucial role in malware research by providing a controlled environment to safely analyze and observe malware behavior. A virtual machine allows researchers to execute malware samples in isolation from the host system, preventing the malware from causing damage or spreading beyond the sandboxed environment. VMs replicate the functions of physical machines, allowing malware to believe it is operating in a legitimate environment, which can lead to the revelation of its true behaviors.

In malware research, virtual machines are commonly used for:

- **Dynamic Malware Analysis:** By running malware in a virtualized environment, researchers can observe how it interacts with the operating system, what files it modifies, what processes it starts, and whether it connects to external servers.
- **Snapshot and Rollback Capabilities:** VMs enable researchers to take snapshots of the system at any point, allowing them to revert to a clean state quickly. This is particularly useful in testing how different variants of malware behave or testing the effects of multiple samples without the need to reinstall the system.
- **Safeguarded Experimentation:** The isolation provided by VMs ensures that malware cannot escape and affect other parts of the network or host machine. This safe environment is essential for analyzing dangerous malware like ransomware or rootkits.

2.7. Memory Acquisition Tools

Memory acquisition is the process of capturing the contents of a system's volatile memory (RAM) for analysis. This is a crucial step in memory forensics, as malicious processes and hidden malware often reside in memory rather than on disk. Since volatile memory is lost once a system is powered down, timely acquisition is essential for preserving evidence in an investigation. Memory acquisition techniques involve using specialized tools to capture a memory snapshot, which can then be analyzed to uncover hidden processes, network connections, or malicious code. These techniques are often used in incident response and malware analysis to provide deeper insights into a system's state during a security breach.

2.7.1. Winpmem

Winpmem is a widely used memory acquisition tool designed for Windows systems. It enables live memory acquisition by capturing the contents of physical memory and creating a memory dump that can be analyzed using forensic tools like Volatility. Some key features of Winpmem include:

- **Live Capture:** Winpmem allows for the capture of memory from a live system without disrupting its normal operation, making it ideal for incident response and ongoing investigations.
- **Support for Different Output Formats:** Winpmem can output memory dumps in multiple formats, such as raw or AFF4 (Advanced Forensic File Format), which are compatible with a variety of forensic analysis tools.
- **Low Impact on System Performance:** The tool is designed to minimize its impact on system performance during memory acquisition, ensuring that it does not significantly alter the system state while acquiring memory.

Winpmem is especially useful for capturing volatile data like active processes, open files, and network connections that would otherwise be lost after a system shutdown or reboot.

2.7.2. Other Memory Acquisition Tools

In addition to Winpmem, several other tools are commonly used for memory acquisition across different platforms:

- **FTK Imager:** Primarily known for its disk imaging capabilities, FTK Imager can also be used to capture memory from live systems. It is a versatile tool widely adopted by forensic investigators for both disk and memory acquisitions.
- **DumpIt:** A simple, user-friendly tool for capturing live memory from Windows systems. DumpIt is favored for its ease of use, often employed in rapid response situations where time is critical.
- **Belkasoft Live RAM Capturer:** This tool is designed for the acquisition of volatile memory on Windows systems, even with advanced anti-debugging and anti-acquisition techniques employed by malware.
- **LiME (Linux Memory Extractor):** A memory acquisition tool specifically designed for Linux systems, LiME enables the acquisition of volatile memory to a dump file that can be analyzed later. It is commonly used in incident response on Linux-based infrastructures.

Each of these tools serves different purposes and may be more suitable for specific platforms or investigative needs.

2.7.3. Challenges in Memory Acquisition

While memory acquisition is a critical part of memory forensics, it comes with several challenges:

- **Volatile Nature of Memory:** Since RAM is volatile, its contents are lost when the system powers down or restarts. This means that investigators must capture memory while the system is still running, which poses the risk of altering the system's state during the acquisition process.
- **Anti-Forensic Techniques:** Some sophisticated malware employs anti-forensic measures to prevent or hinder memory acquisition. For example, malware might detect memory acquisition tools and terminate itself or modify memory contents to avoid detection.
- **Performance Overhead:** The process of acquiring memory from a live system can cause a performance overhead, potentially affecting the system's behavior or corrupting the very evidence investigators are trying to preserve.

- **Data Integrity:** Ensuring the integrity of the acquired memory is another major challenge. The acquisition process should not alter or contaminate the memory dump, and forensic tools need to maintain a clear chain of custody to ensure that evidence can be validated in court or during an investigation.

Despite these challenges, memory acquisition remains a vital step in forensic investigations, providing a wealth of information about a system's state during an attack or breach.

2.8. Overview of Relevant Literature

Sleep obfuscation has been recognized as a growing challenge in malware analysis. Traditional tools like antivirus software and basic memory analysis techniques often fail to detect malware employing sleep obfuscation, as these techniques use deliberate delays or dormant periods to evade detection [4]. Research indicates that while solutions like Windows Defender and VirusTotal provide a first line of defense, they are largely signature-based and struggle against more advanced evasion tactics. Studies have shown that more dynamic approaches, such as CPU instruction emulation and deep memory scanning, offer a higher chance of identifying obfuscated behavior. Volatility's malfind plugin, for instance, attempts to detect hidden processes in memory, but its limitations become evident when dealing with delayed execution or sleep-based obfuscation [4].

To address these gaps, recent research has explored the use of emulation and advanced memory analysis to detect sophisticated malware behavior [5]. However, many of these methods either lack scalability or require extensive manual intervention. Projects utilizing tools like Unicorn for CPU emulation have demonstrated success in identifying hidden shellcode, but their application has primarily focused on post-mortem analysis of memory dumps. This highlights the need for more comprehensive solutions that can detect such evasive techniques both in static and live environments, which positions the MED project as a relevant contribution to this ongoing area of research.

3. Research

3.1. Malware Research

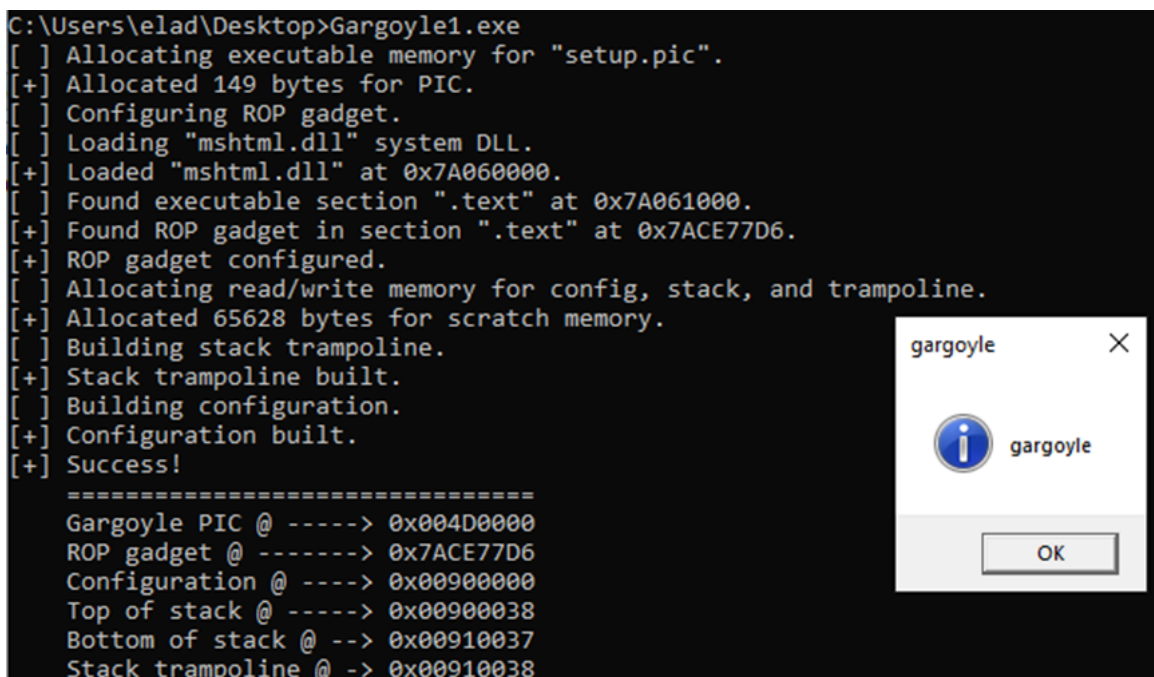
In this section we will cover a representative sample of the sleep-obfuscators tools, discuss the common factor among these tools, and from this common factor we will derive our detection solution.

3.1.1. Gargoyle

Gargoyle[6] is considered to be the pioneer of sleep-obfuscators, developed by Josh Lospinoso in 2017.

Gargoyle Running Flow

In this case, just to be safe, the payload of Gargoyle is only a message box pop up.



```
C:\Users\elad\Desktop>Gargoyle1.exe
[ ] Allocating executable memory for "setup.pic".
[+] Allocated 149 bytes for PIC.
[ ] Configuring ROP gadget.
[ ] Loading "mshtml.dll" system DLL.
[+] Loaded "mshtml.dll" at 0x7A060000.
[ ] Found executable section ".text" at 0x7A061000.
[+] Found ROP gadget in section ".text" at 0x7ACE77D6.
[+] ROP gadget configured.
[ ] Allocating read/write memory for config, stack, and trampoline.
[+] Allocated 65628 bytes for scratch memory.
[ ] Building stack trampoline.
[+] Stack trampoline built.
[ ] Building configuration.
[+] Configuration built.
[+] Success!
=====
Gargoyle PIC @ -----> 0x004D0000
ROP gadget @ -----> 0x7ACE77D6
Configuration @ ----> 0x00900000
Top of stack @ -----> 0x00900038
Bottom of stack @ --> 0x00910037
Stack trampoline @ -> 0x00910038
```

Figure 1: Gargoyle is executing and running its payload.

When gargoyle is sleeping, it marks the payload pages as read only:

Base address	Type	Size	Protect...
0x4d0000	Private: Commit	4 kB	R

Figure 2: Gargoyle page protection Read

While the payload is running, the pages are marked back to read execute:

Base address	Type	Size	Protect...
0x4d0000	Private: Commit	4 kB	RX

Figure 3: Gargoyle page protection Read Execute

Gargoyle mechanism - high level

1. Change the payload memory region protections to R.
2. Sleep for a while.
3. Change back the payload memory region protections to RX.
4. Run payload.
5. Go to 1.

How does it work?

First of all, it's important to emphasize two critical points:

1. As mentioned in the high-level description of Gargoyle, obfuscation is not used in this technique. However, Gargoyle has established the foundational principles for all subsequent sleep-obfuscators and embodies the key fundamentals of a sleep-obfuscator.
2. As we will see later, Gargoyle's operation is based on a Return-Oriented Programming (ROP) technique, which relies on a stack-based calling convention. This makes it more relevant for 32-bit systems. However, there is also a 64-bit equivalent implementation, which we will not cover here, as the fundamental principles are similar.

Gargoyle's mechanism:

Before going to sleep, Gargoyle removes the execution permission from the payload's memory, then it waits on a timer.

The timer has two important fields:

1. Completion routine which would run when the timer expires.

The completion routine it configures is a ROP chain the looks like this:

```
pop ecx
pop esp
ret
```

The purpose of this ROP chain is that the “ret” instruction in the end would actually jump to the address that is saved in the argument passed to the completion routine.

That is because the completion routine's argument resides 4 bytes from the top of the stack (esp+4), so when we execute pop ecx the esp moves forward 4 bytes, and when we execute pop esp, the value of the argument is loaded into esp. Then when we execute the ret instruction, we actually jump to the address that the completion routine's parameter points to.

That way we have full control on what will be executed next.

2. The completion routine's argument.

In this case, Gargoyle configures the argument to be the following custom stack:

```
struct StackTrampoline {
void* VirtualProtectEx; // <-- ESP here; ROP gadget rets
void* return_address; // Gargoyle payload's address to return to after calling
VirtualProtectEx.
void* current_process; // First arg to VirtualProtectEx
void* address;          // Payload's address.
uint32_t size;
uint32_t protections; // RX protections.
void* old_protections_ptr;
uint32_t old_protections; // Last arg to VirtualProtectEx
void* setup_config;     // Some Gargoyle configuration
};
```

With this custom stack, everything is ready for execution.

Putting it all together, this is the running flow:

1. Construct a custom stack containing VirtualProtectEx (address and parameters) and the payload's address.
2. Loading the dll that contains the ROP gadget into memory.
3. Set a timer with the ROP gadget set as completion routine , and custom stack set as argument.
4. Timer expires and invokes ROP gadget.
5. After ROP gadget execution, esp points to the custom stack, and the ret instruction calls VirtualProtectEx.
6. After VirtualProtectEx executes and adds execute permission to the payload's memory region, it returns to the payload.
7. Payload executes.
8. Back to step 3.

3.1.2. Cronos

Cronos[7] is a sophisticated sleep-obfuscator developed by Ido Veltzman in 2022, inspired by some older but simpler sleep-obfuscators like Gargoyle.

Cronos Running Flow

In this case, just to be safe, the payload of cronos is only a message box pop up.

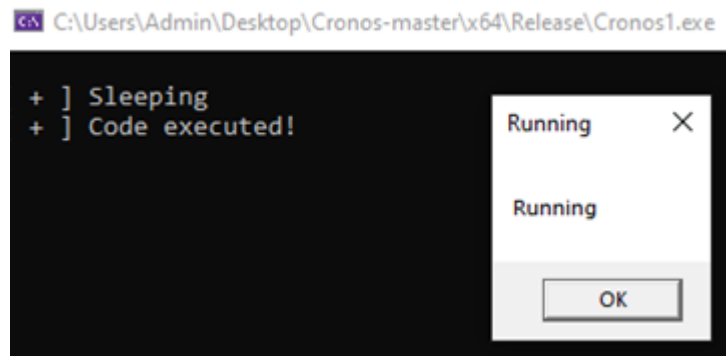


Figure 4: Cronos is executing and running its payload.

You can see that while running the main executable's memory region marked as RWX:

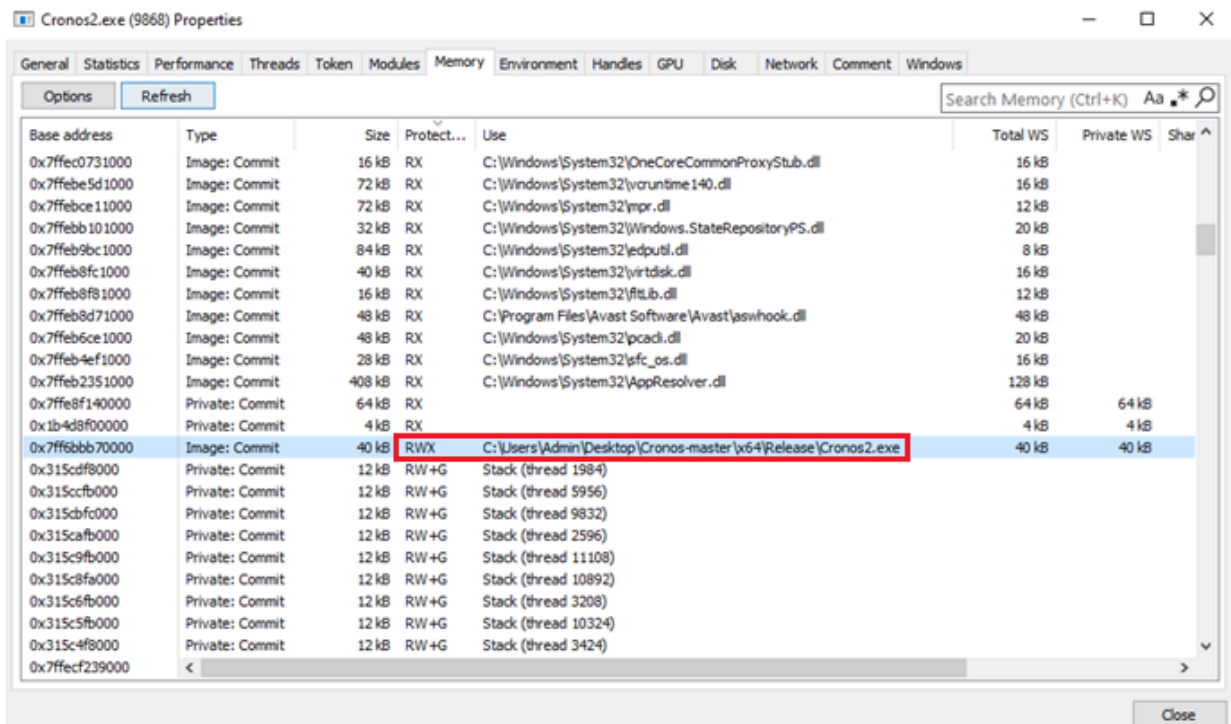


Figure 5: Cronos memory while executing.

And while it sleeps it removes the executable permission:

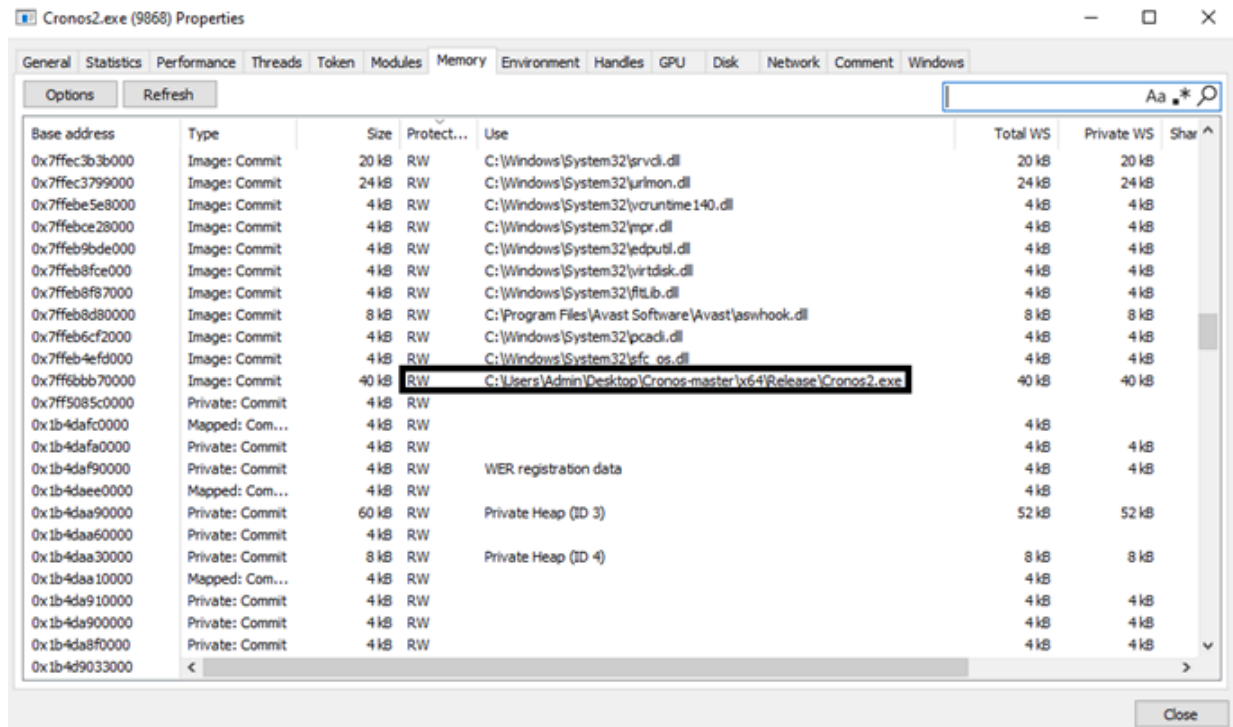


Figure 6: Cronos memory while sleeping.

Cronos mechanism - high level

1. Change the image's protection to RW.
2. Encrypt the image.
3. Sleep for a while.
4. Decrypt the image.
5. Add execution protection to the image's memory.
6. Run payload.
7. Go to 1.

How does it work?

Everything is built on top of the undocumented `NtContinue`[8] function, which is a function that gets as a parameter a thread context, and uses it to invoke the thread's execution.

For the encryption and decryption operations it uses the undocumented function `SystemFunction032`[9] that gets a buffer and a key, and encrypts/decrypts the data using RC4.

For the protection changes (RW\RWX) it uses the traditional and well documented VirtualProtect function.

So, for every one of the four stages (which are: change protections to RW, encryption, decryption and change protections to RWX, as described above), it builds a thread context. The thread context includes the next instruction to run (which is NtContinue or SystemFunction032) in the RIP register, and other parameters (like encryption key, memory address, protections etc.), depending on the specific operation.

```
// VirtualProtect( ImageBase, ImageSize, PAGE_READWRITE, &OldProtect );
ctxProtectionRW.Rsp -= (8 + 0x150);
ctxProtectionRW.Rip = (DWORD_PTR) VirtualProtect;
ctxProtectionRW.Rcx = (DWORD_PTR) ImageBase;
ctxProtectionRW.Rdx = ImageSize;
ctxProtectionRW.R8 = PAGE_READWRITE;
ctxProtectionRW.R9 = (DWORD_PTR) &oldProtect;

ctxEncryption.Rsp -= (8 + 0xF0);
ctxEncryption.Rip = (DWORD_PTR) SystemFunction032;
ctxEncryption.Rcx = (DWORD_PTR) &Image;
ctxEncryption.Rdx = (DWORD_PTR) &Key;

ctxDecryption.Rsp -= (8 + 0x90);
ctxDecryption.Rip = (DWORD_PTR) SystemFunction032;
ctxDecryption.Rcx = (DWORD_PTR) &Image;
ctxDecryption.Rdx = (DWORD_PTR) &Key;

ctxProtectionRWX.Rsp -= (8 + 0x30);
ctxProtectionRWX.Rip = (DWORD_PTR) VirtualProtect;
ctxProtectionRWX.Rcx = (DWORD_PTR) ImageBase;
ctxProtectionRWX.Rdx = ImageSize;
ctxProtectionRWX.R8 = PAGE_EXECUTE_READWRITE;
ctxProtectionRWX.R9 = (DWORD_PTR) &oldProtect;
```

Figure 7: Building thread contexts, one per stage.

Then, for every stage it initializes a waitable timer whose completion routine is NtContinue, while the parameter of the completion routine (NtContinue) is the thread context of that stage. The time of execution is according to the operation (change to RW and encryption are done immediately, and the decryption + change to RWX are done at the end of the sleep time).

```
InitializeTimerMs(&protectionRWDueTime,    0 );  
InitializeTimerMs(&encryptionDueTime,      1 );  
InitializeTimerMs(&decryptionDueTime,      sleepTime-1 );  
InitializeTimerMs(&protectionRWXDueTime,   sleepTime);
```

Figure 8: Timer initialization of each stage.

That way, using waitable timers and thread contexts, it actually builds a sleep-obfuscation execution flow that can run independently and even while the actual payload cannot be executed (because it doesn't have the execute permission).

3.1.3. Ekko

The Ekko[10] tool, developed by Cracked5pider, is a proof-of-concept that employs a memory evasion technique using the CreateTimerQueueTimer Windows API. It was designed to demonstrate how timers can be leveraged to obfuscate malware's presence by manipulating execution delays, which makes traditional detection methods less effective.

This tool highlights the increasing sophistication of memory evasion techniques, where subtle manipulations of system functions can be used to evade detection, further underscoring the need for advanced analysis tools like the one developed in this project.

3.2. Common factor

The following figure demonstrates the common factor of all sleep-obfuscators. The key point is that most of the time they stay hard to detect and stealthy, by being non executable and optionally obfuscated.

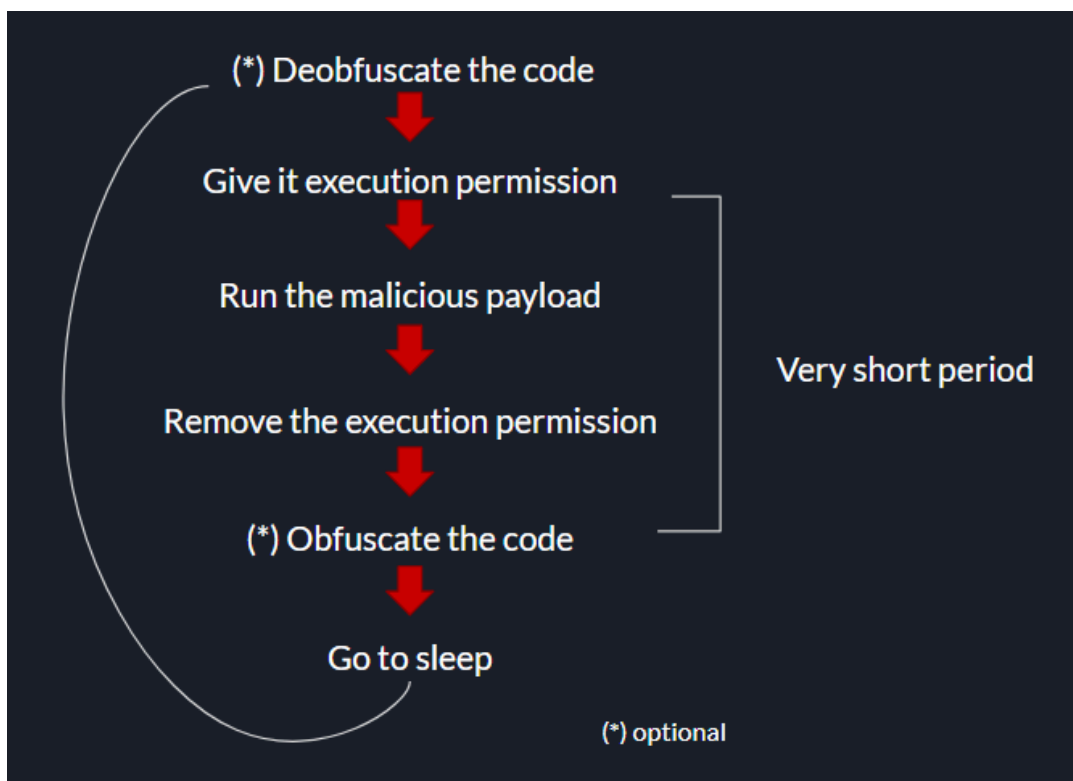


Figure 9: General sleep-obfuscator execution flow.

3.3. Offline Solution

The main problem that traditional memory scanners face while trying to catch sleep-obfuscators is that they are capable of catching it only when it's running.

However, as we already mentioned before, most of the time sleep-obfuscators are not running nor runnable. This makes them most of the time undetectable by AVs or other memory scanners.

For this reason, we tried another approach, which is trying to catch sleep-obfuscators in its usual state, meaning while it's sleeping.

Just to remind, when a sleep-obfuscator is sleeping, it waits for a timer that would eventually make the payload executable and run it in some way.

Therefore, our solution involves parsing all timers in the system and for each timer conducting CPU emulation (using Unicorn emulator) of the timer's completion routine.

If our emulation shows that the completion routine eventually triggers a VirtualProtect or VirtualProtectEx call (which used to make a memory region executable), we mark the memory region as suspicious and dump it.

Our solution was implemented on top of Volatility Framework, which is a convenient framework for working with static memory dumps.

In the following figure you can see a side-by-side comparison of the sleep-obfuscator execution flow and our detection solution:

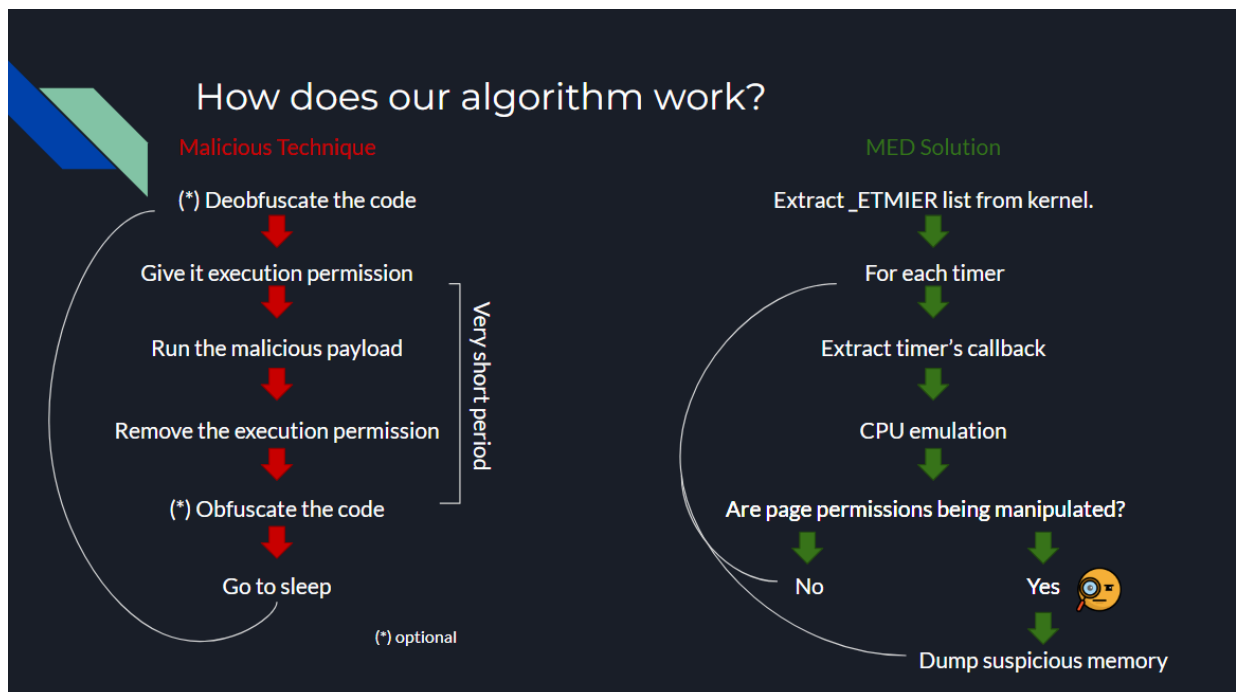


Figure 10: General sleep-obfuscator execution flow vs MED's algorithm.

3.4. Live Solution

To apply the solution on live machines, we utilized the **WinPmem driver**, an open-source physical memory acquisition tool for Windows. This tool allows **Volatility**—originally designed to work with static memory dumps—to access the real-time memory of a host machine as if it were a static file. This setup enables the MED project to apply the same analysis techniques used on static dumps to live systems, enhancing its versatility and effectiveness.

4. System Design and Implementation

4.1. System Architecture

The MED project's architecture has three main components.

4.1.1. Volatility Framework

The core of the project is the Volatility Framework, a powerful tool used to read and parse memory structures from memory images. This allows for detailed analysis of the system's memory to uncover potential issues or malicious activity.

4.1.2. Unicorn CPU Emulator

The second critical component is the Unicorn CPU emulator, utilized to emulate the timer structures identified through Volatility. By doing so, it verifies whether these structures are designed to alter memory page protections, potentially incriminating them.

4.1.3. WinPmem Driver

The final component is the WinPmem driver, an open-source physical memory acquisition tool for Windows. We used this tool to capture the memory of live machines, enabling the detection of sleep obfuscators in both live environments and memory dumps.

These three components form a seamless pipeline for the MED project. First, **WinPmem** acquires memory data from a live system or dump. This data is then processed by **Volatility**, which parses it for further analysis. Finally, **Unicorn** emulates the parsed data to identify potential sleep obfuscators. The results are reported back to the user, who can investigate the suspicious processes further and even upload the suspected binary to **VirusTotal** for additional scrutiny.

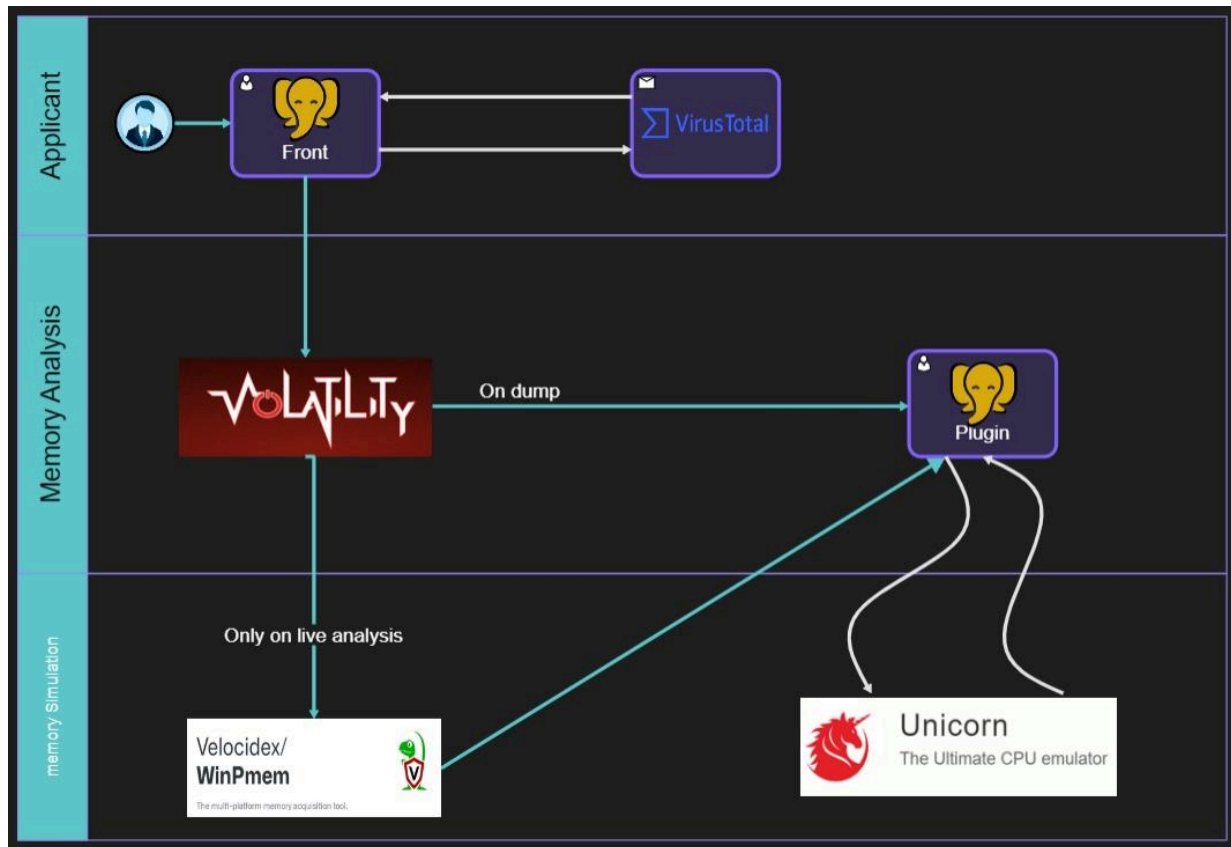


Figure 11: Complete Architecture

4.2. Implementation Details

The MED project is implemented in Python, designed to be modular and scalable. At its core, an abstract class “ScreenMgmt” manages the system's screens. Each ‘screen’[Figure 12] extends this class, enabling new functionalities to be added easily while maintaining a consistent interface.

A custom plugin was built for the Volatility framework to perform memory scanning and analysis. This plugin uses the **Unicorn library** to emulate CPU instructions, providing enhanced analysis by simulating the behavior of malicious code within the memory being scanned. This combination of Volatility and Unicorn allows for deeper investigation of memory contents.

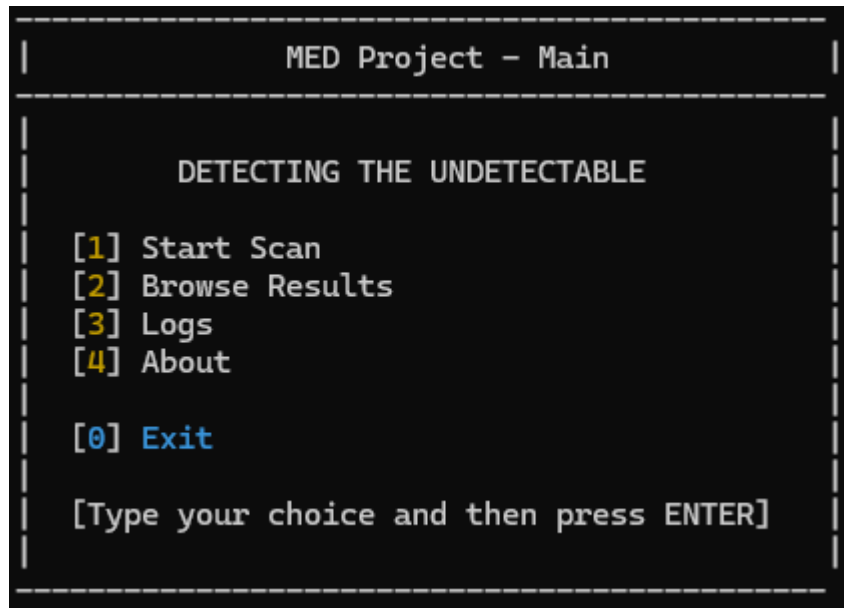


Figure 12: 'Screen Main' inherits ScreenMgmt abstract class

The system supports two key features:

1. **VirusTotal Integration** [Figure 13]: Extracted malicious data is sent to VirusTotal for antivirus scanning results.
2. **Memory Peeking** [Figure 14]: The beginning of extracted memory can be inspected for initial analysis.

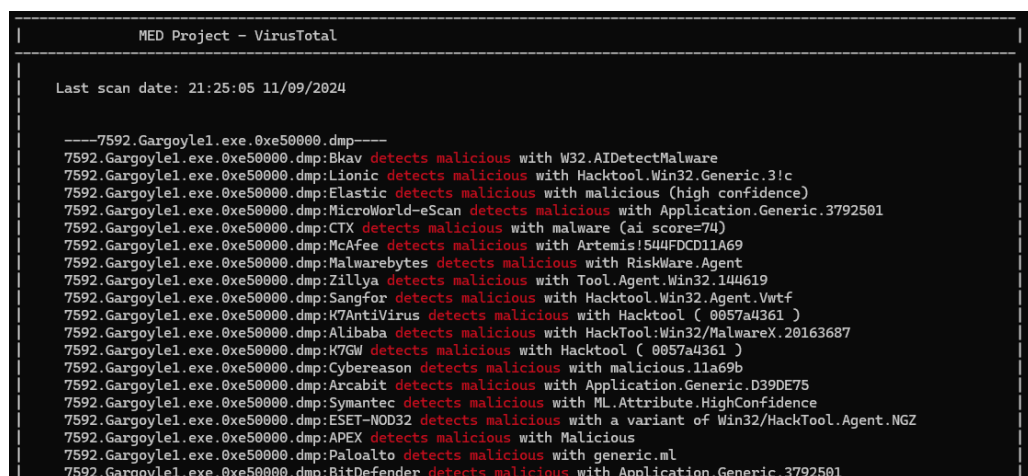


Figure 13: VT integration

```

MED Project - Peek

Process: Gargoyle1.exe, Pid: 7592, Address: 14942208

8b 5c 24 04 8d a3 34 00 01 00 89 e5 8b 13 83 fa .\$.4.....
00 75 2f 6a 00 6a 00 6a 00 8b 4b 14 ff d1 89 43 .u/j.j.j..K....C
24 6a 00 8b 4b 20 51 8b 4b 2c 51 8b 4b 28 51 8d $j..K Q.K,Q.K(Q.
4b 30 51 8b 4b 24 51 8b 4b 18 ff d1 c7 03 01 00 K0Q.K$Q.K.....
00 00 8b 4b 0c 89 8b 38 00 01 00 6a 00 68 6f 79 ...K...8...j.hoy
6c 65 68 67 61 72 67 89 e1 6a 40 51 51 6a 00 8b lehgarg..j@QQj..
4b 1c ff d1 89 ec 6a 01 6a ff 8b 4b 24 51 6a 00 K....j.j..K$Qj..
6a 01 6a ff 8b 4b 24 51 8b 4b 10 51 8d 4b 30 51 j..K$Q.K.Q.K0Q
6a 02 8b 4b 08 51 8b 4b 04 51 6a ff 8b 4b 10 51 j..K.Q.K.Qj..K.Q
8b 4b 0c ff e1 00 00 00 00 00 00 00 00 00 00 .K.....
00 00 00 00 00 .....

0xe40000:  mov     ebx, dword ptr [esp + 4]
0xe40004:  lea     esp, [ebx + 0x10034]
0xe4000a:  mov     ebp, esp
0xe4000c:  mov     edx, dword ptr [ebx]
0xe4000e:  cmp     edx, 0
0xe40011:  jne     0xe40042
0xe40013:  push    0
0xe40015:  push    0
0xe40017:  push    0

```

Figure 14: Memory Peeking.

In cases where live memory scans are required, the system uses ‘winpmem’ to read live memory data directly from the system. This provides a powerful capability for analyzing live systems, complementing the analysis of memory dumps handled by Volatility.

The system was tested on a Windows 10 machine with 8GB of RAM processor.

Key tools include:

- **Python 3.12:** Core language.
- **Volatility Framework & Unicorn:** For memory analysis and CPU emulation.
- **Winpmem:** For live memory capture.
- **VirusTotal API:** For antivirus scanning.

4.3. Evaluation Metrics

As previously noted none of the existing mitigations static (Volatility’s Malfind) or live (Anti viruses) could detect those sleeping obfuscators. In order to evaluate our solution we used the MED project on various virtual machines with different architectures and used various different sleeping obfuscators and found out that our algorithm always (live or static) found the sleep obfuscators without a single false positive.

In addition we used the project's feature to send the shellcode and the malicious binary to VirusTotal which even when handed on a silver plate could only detect Gargoyles sleep obfuscator but couldn't detect Cronos.

```
MED Project - VirusTotal

Last scan date: 23:27:42 31/08/2024

----6596.Cronos1.exe.0x7ff7245c0000.dmp----
6596.Cronos1.exe.0x7ff7245c0000.dmp: no detection found

----pid.6596.vad.0x7ff7245c0000-0x7ff7245c8fff.dmp----
pid.6596.vad.0x7ff7245c0000-0x7ff7245c8fff.dmp: no detection found

[1] Rescan
[2] Return

[0] Return to Menu

[Type your choice and then press ENTER]
```

Figure 15: VirusTotal results on Cronos binary and shellcode.

```
MED 2024-08-31

7592.Gargoyl1.exe.0xe50000.dmp:Zillya detects malicious with Tool.Agent.Win32.144619
7592.Gargoyl1.exe.0xe50000.dmp:McAfeeD detects malicious with tildAD3BFE6F0A4
7592.Gargoyl1.exe.0xe50000.dmp:Trapmine detects malicious with suspicious.low.ml.score
7592.Gargoyl1.exe.0xe50000.dmp:Sophos detects malicious with Generic.Reputation.PUA (PUA)
7592.Gargoyl1.exe.0xe50000.dmp:Ikarus detects malicious with Trojan.Win32.HackTool
7592.Gargoyl1.exe.0xe50000.dmp:Varist detects malicious with W32/ABApplication.JINY-5458
7592.Gargoyl1.exe.0xe50000.dmp:Avira detects malicious with TR/Hacktool.gyfwy
7592.Gargoyl1.exe.0xe50000.dmp:Arcabit detects malicious with Application.Generic.D39DE75
7592.Gargoyl1.exe.0xe50000.dmp:GData detects malicious with Application.Generic.3792501
7592.Gargoyl1.exe.0xe50000.dmp:Google detects malicious with Detected
7592.Gargoyl1.exe.0xe50000.dmp:ALYac detects malicious with Application.Generic.3792501
7592.Gargoyl1.exe.0xe50000.dmp:MAX detects malicious with malware (ai score=74)
7592.Gargoyl1.exe.0xe50000.dmp:Cylance detects malicious with Unsafe
7592.Gargoyl1.exe.0xe50000.dmp:Rising detects malicious with HackTool.Agent!8.335 (TFE:5:g5MDRpU3FLK)
7592.Gargoyl1.exe.0xe50000.dmp:Fortinet detects malicious with W32/Agent.NGZ!tr
7592.Gargoyl1.exe.0xe50000.dmp:DeepInstinct detects malicious with MALICIOUS
7592.Gargoyl1.exe.0xe50000.dmp:CrowdStrike detects malicious with win/malicious_confidence_90% (D)
7592.Gargoyl1.exe.0xe50000.dmp:alibabacloud detects malicious with HackTool:Win/Gyfwy.Gen

----pid.7592.vad.0xe40000-0xe40fff.dmp----
pid.7592.vad.0xe40000-0xe40fff.dmp: no detection found

[1] Rescan
[2] Return

[0] Return to Menu

[Type your choice and then press ENTER]
```

Figure 16: VirusTotal results on Gargoyles binary and shellcode.

5. Results and Analysis

5.1. Experimental Setup

In the realm of cybersecurity research, particularly when dealing with malware experiments, it is crucial to maintain a safe and controlled environment. To achieve this, we employed virtual machines (VMs) as our primary experimental platform. The use of VMs allowed us to effectively isolate and contain any potentially harmful activities associated with malware execution. This isolation is vital because it prevents the malware from interacting with the host system or spreading to other environments, which could lead to widespread contamination.

Moreover, VMs offer a unique advantage: they can be reverted to clean snapshots after each test. This capability not only enhances security but also ensures the reproducibility of our experiments. By restoring to a known clean state, we eliminate any residual effects from previous tests that might skew the results. [Refer to Figure 17 for a visual representation of this setup.]

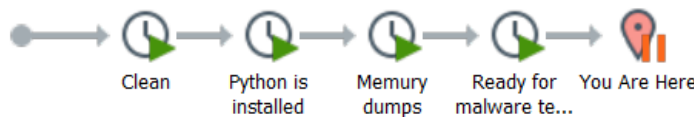


Figure 17: Working with VM snapshots

The use of VM snapshots provided us with a double benefit: for static memory analysis, we could work with the snapshot's memory dump, ensuring a constant environment for our experiments. This allowed us to obtain real and reliable results without the interference of redundant variables. For live memory analysis, we could revert to the VM snapshot whenever we needed to examine the malware and the tools we used, maintaining a relatively static environment with minimal extraneous variables.

To further enhance our experimental security, we utilized Windows Defender within the virtual machines. This provided a baseline level of protection and allowed us to demonstrate the

limitations of standard antivirus software when faced with sophisticated malware techniques. In addition to Windows Defender, we extended our testing to other antivirus solutions, including Avast, to obtain a comprehensive overview of detection efficacy across different platforms.

Our analysis also included a thorough examination of memory behavior using the Volatility Framework. This powerful tool was configured with a set of generic plugins that facilitated detailed investigations into malware behavior through memory dumps. By analyzing these memory states, we could observe how the malware interacted with system resources and the overall environment. All experiments were conducted on a machine equipped with 8GB of RAM.

To maintain consistency and reliability, we ensured that all VM snapshots and configurations were meticulously documented, allowing for easy replication of the experimental conditions.

5.2. Presentation of Results

Throughout the course of our experiments, we rigorously and systematically executed the **Malware Evasion Detection (MED)** tool against a comprehensive range of researched malware samples. These tests were conducted in both **static** and **live environments**, providing a well-rounded evaluation of MED's capabilities under various conditions. The static analysis involved examining memory dumps and analyzing the system's state without active interaction, while live testing allowed us to observe how the malware behaved in real time, as it executed and attempted to evade detection mechanisms.

The results of these experiments were overwhelmingly positive, clearly demonstrating the robustness, precision, and efficiency of the MED tool in detecting malicious activities, even when faced with complex, evasive malware techniques. In every tested scenario, regardless of the specific malware or the environment in which it was analyzed, MED performed consistently and effectively. The tool successfully identified the presence of malware, accurately locating and extracting the relevant memory pages associated with the malicious processes.

A key strength of MED lies in its ability to go beyond mere detection—it also retrieves and analyzes the **shellcode** executed by the malware. Shellcode is the essential code used by malware to gain control over a system, and the ability to extract this code provides crucial insights into the behavior and objectives of the malware. By capturing this shellcode from memory, MED allows for an in-depth investigation into the precise operations that the malware performs once it gains a foothold within the system.

This level of detection, combined with the ability to extract actionable data such as memory pages and shellcode, sets MED apart from more conventional malware detection tools. The tool's advanced scanning algorithms and memory analysis capabilities were able to detect even the most elusive threats, including those employing sophisticated evasion techniques such as sleep obfuscation.

The consistent performance of MED across various scenarios, malware types, and testing environments is illustrated in **Figures 18 and 19**. These figures showcase specific instances of detection, with **Figure 18** displaying a live scan of the **Gargoyle** malware sample, where MED successfully identified the threat and captured its activity in real time. **Figure 19** highlights a

static scan of the **Cronos** malware sample, demonstrating MED's ability to effectively analyze and extract malicious activity from memory dumps. These figures underscore MED's versatility and reliability in detecting and analyzing malware, regardless of the testing conditions.

```
|-----|
| MED Project - Results |
|-----|
| Malicious activity detected |
|
| - Process: Gargoyle1.exe
| - PID: 7592
| - Address: 14942208
| - Protection: PAGE_READWRITE
| - Vad Tag: VadS
|
| [1] Upload results to VT
| [2] Peek
|
| [0] Return to Menu
|
| [Type your choice and then press ENTER]
|-----|
```

Figure 18: Live scan detecting Gargoyle.

```
|-----|
| MED Project - Results |
|-----|
| Malicious activity detected |
|
| - Process: Cronos1.exe
| - PID: 6596
| - Address: 140699443658752
| - Protection: PAGE_EXECUTE_WRITECOPY
| - Vad Tag: Vad
|
| [1] Upload results to VT
| [2] Peek
|
| [0] Return to Menu
|
| [Type your choice and then press ENTER]
|-----|
```

Figure 19: Static scan detecting Cronos.

In addition to its capabilities in detecting a variety of malware samples, we conducted a thorough evaluation of the **Malware Evasion Detection (MED)** tool in both **static** and **live environments** to ensure that it not only excels in identifying malicious activity but also maintains a high level of precision by minimizing false positives. A detection tool is only as effective as its ability to accurately distinguish between real threats and benign system behavior. An overly sensitive tool that triggers frequent false positives can lead to alarm fatigue, causing analysts to overlook or dismiss genuine threats. Therefore, it was essential that MED demonstrated the ability to reliably differentiate between malware and normal, non-malicious system operations.

To assess this, we included **clean machines** (systems that had not been infected by malware) as part of our testing suite. These clean environments provided a critical baseline, allowing us to determine whether MED could accurately distinguish between legitimate, everyday processes and potentially harmful activity. The tests conducted on these clean machines were designed to simulate typical system usage, including common processes and applications, which would be expected to run on any uninfected system. This simulated real-world scenarios where distinguishing normal system behavior from malicious activity is crucial.

The results of these tests were highly encouraging. MED proved to be a highly discerning tool, as it successfully avoided generating false alarms during the analysis of clean machines. Unlike many overly generic detection tools that rely on broad heuristics and may trigger alerts for benign processes, MED demonstrated a nuanced understanding of system activity. It effectively ignored routine processes and benign system behaviors, while still maintaining a sharp focus on identifying actual threats when they were present.

This precision is critical to the practicality of MED in a real-world cybersecurity environment. False positives can be a significant challenge for cybersecurity teams, leading to wasted time and resources chasing down non-existent threats. A tool that minimizes false positives, like MED, allows security analysts to focus their attention on genuine risks, improving both the efficiency and effectiveness of incident response.

The effectiveness of MED in avoiding false positives is illustrated in the following figure, where running the tool on a clean machine resulted in a reassuring message, clearly indicating that no malicious activity was detected. This confirmation of a "clean" system demonstrates that MED is not overly sensitive or prone to false alarms, providing confidence that the tool can

be trusted to accurately report malware without triggering unnecessary alerts. This level of accuracy in distinguishing between infected and uninfected systems enhances the reliability and overall utility of the tool in diverse environments.

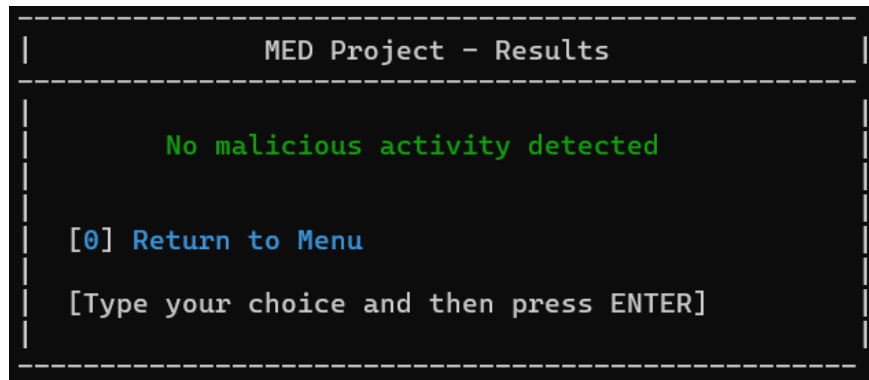


Figure 20: Running a scan on a clean machine.

In the section “Comparison with Existing Approaches” below you can see a chart that shows how MED performed against some tricky and sophisticated sleep-obfuscators.

5.3. Data Analysis and Interpretation

Our experiments yielded compelling evidence that MED is particularly effective in detecting **sleep obfuscators** - an advanced technique employed by malware to delay execution and avoid detection by traditional antivirus tools. Many conventional security solutions struggle with these delayed behaviors, as they often rely on immediate and overt signs of malicious activity.

In contrast, MED's advanced capabilities in memory scanning and CPU emulation enabled it to capture and analyze sleep obfuscation techniques effectively. This allowed for the extraction of related shellcode, providing valuable insights into the malware's behavior.

Throughout all tested scenarios, MED consistently identified sleep obfuscators, offering a more comprehensive understanding of the malware's operational patterns than traditional tools. The ability to detect these evasive tactics represents a significant advantage over conventional approaches, which frequently fail to recognize the subtleties of such advanced obfuscation methods.

Additionally, MED reliably avoided generating false-positive results in all clean static and live memory scenarios. This accuracy not only saves researchers valuable time but also prevents unnecessary alarm for users.

As a result, MED stands out as a dependable and thorough solution in addressing the challenges posed by sleep obfuscator malware.

5.4. Comparison with Existing Approaches

In our quest to evaluate the efficacy of MED, we compared it against established tools such as Volatility’s ‘malfind’ plugin, VirusTotal (VT), Windows Defender and Avast.

The comparison revealed a distinct advantage for MED in its ability to detect sleep obfuscators—techniques that many traditional tools often overlook. While both ‘malfind’ and VirusTotal excel in flagging more obvious malicious behaviors, they falter when confronted with sophisticated malware employing sleep obfuscation tactics. Windows Defender and Avast, although competent at identifying common malware signatures, similarly struggle with advanced obfuscation techniques that are designed specifically to evade detection.

MED’s primary strength lies in its specialized ability to detect and analyze sleep obfuscators, effectively filling a critical gap left by traditional tools. However, it is important to note that MED is focused primarily on this specific type of obfuscation, whereas tools like VirusTotal and Windows Defender offer broader malware detection capabilities, including signature-based and heuristic methods.

Below, we present a chart that compares the detection capabilities of each tool against various malware samples, highlighting the strengths and weaknesses of each:

	malfind	Windows Defender	Avast	VirusTotal*	MED
Gargoyle					
Cronos					
Ekko					

Figure 21: Detection Comparison

The chart highlights MED’s ability to consistently detect sleep obfuscators (even when traditional tools like ‘malfind’ and Defender miss them). On the other hand, general-purpose tools like VT and Defender perform better across a broader range of malware types, though they lack MED’s specialization in detecting sleep obfuscators.

***Note:** As mentioned briefly earlier (see “Implementation Details” section) and explained here, VirusTotal has certain advantages over the other defensive tools for several reasons:

- In terms of static file system analysis, while Windows Defender and Avast operate on a live operating system with numerous files, making it significantly more complex to identify a single malicious file, VirusTotal is provided the malware file directly by the MED tool.

However, it is important to highlight that even when we manually ran scans with Windows Defender or Avast on the specific malware file, they still failed to classify it as malicious. Although this is not the main focus of our experiment, which centers on memory analysis of sleep-obfuscation tools, it is a noteworthy point.

- In terms of static and live memory analysis, while all tools contend with a vast amount of data, some of which can be misleading due to numerous processes and modules, VirusTotal again benefits from having the malware's memory payload directly provided by the MED tool.
- It's important to note that VirusTotal aggregates results from a wide range of antivirus solutions, EDRs, and other tools, rather than functioning as a standalone product. This makes it potentially one of the most comprehensive tools for comparison.

Even with these considerations, the chart above clearly shows that VirusTotal is outperformed by MED.

5.5. Discussion of Findings

The primary aim of our research was to develop a generalized solution to the "Sleep Obfuscation" technique, a cutting-edge method in the field of malware evasion. As demonstrated by the analysis and comparisons presented in this study, our solution is capable of detecting various types of sleep obfuscators through memory dumps and live system analysis—instances that existing tools fail to identify effectively.

This domain is characterized by an ongoing, dynamic "cat-and-mouse" game between malware developers and security researchers. As a result, it remains possible for new iterations of the "Sleep Obfuscation" method to emerge, potentially capable of evading detection by even our proposed solution. Therefore, it is inevitable that future research will be required to address the next wave of sleep obfuscation techniques, ensuring continued adaptability to the evolving landscape of malware hiding strategies.

6. Conclusion and Future Work

The MED project successfully addressed a critical gap in malware detection by focusing on the identification of sleep obfuscators, a sophisticated evasion technique often missed by traditional tools such as VirusTotal, Windows Defender, and Volatility's 'malfind' plugin. By utilizing deep memory analysis and CPU emulation, MED consistently detected these obfuscators, demonstrating its effectiveness in enhancing the detection of complex malware.

However, the project has some limitations. MED is specifically designed to detect sleep obfuscators and does not yet cover other evasion tactics, such as packers or code injection techniques. Additionally, while MED performed well in controlled environments, its scalability and performance in large-scale real-world scenarios need further investigation.

Future research could focus on expanding MED's capabilities to detect a broader range of obfuscation methods and improving its performance in live environments. Integrating machine learning models to classify and adapt to evolving malware behaviors would also be a promising direction for further development.

In conclusion, MED has made a significant contribution to malware detection by focusing on sleep obfuscators. With further enhancements, it has the potential to become a valuable tool for broader malware analysis in the cybersecurity field.

7. References

- [1] <https://github.com/volatilityfoundation/volatility>
- [2] <https://github.com/unicorn-engine/unicorn>
- [3] M Sikorski & A. Hoing, “MALWARE ANALYSIS IN VIRTUAL MACHINES” in “Practical Malware Analysis”, pp. 29-39
- [4] https://jatayuccloud.app/quackland.tk/let_me_sleep_research.html
- [5] Pythons and Unicorns and Hancitor...Oh My! Decoding Binaries Through Emulation
- [6] <https://github.com/JLospinoso/gargoyle>
- [7] <https://github.com/Idov31/Cronos>
- [8] <http://undocumented.ntinternals.net/index.html?page=UserMode%2FUndocumented%20Functions%2FNT%20Objects%2FThread%2FNtContinue.html>
- [9] <https://osandamalith.com/2022/11/10/encrypting-shellcode-using-systemfunction032-033/>
- [10] <https://github.com/Cracked5pider/Ekko>

8. Appendix A

A.1 Project Setup Instructions

To set up the MED project environment, follow these steps:

1. **Python Version:** Ensure you have Python 3.12 installed on your system.
2. **Install Required Libraries:**

Use the following command to install all dependencies:

```
pip install -r requirements.txt
```

- Key libraries include:
 - Volatility: For memory forensics.
 - Unicorn: For CPU emulation.
 - Requests: For interacting with the VirusTotal API.
- 3. **VirusTotal API Key:**
 - To enable integration with VirusTotal, register for an API key from VirusTotal and add it to the MED configuration file under the `api_key` parameter.